

Received 15 May 2026, accepted 24 May 2026, date of publication 27 May 2026, date of current version 1 June 2026.

Digital Object Identifier 10.1109/ACCESS.2026.3697530

RESEARCH ARTICLE

An AIoT-Based Computer Vision System for Post-Larval Shrimp Detection and Counting in Aquaculture

YO-PING HUANG^{1,2,3}, (Fellow, IEEE), AND IRGI SURYA¹, (Member, IEEE)

¹Department of Electrical Engineering, National Taipei University of Technology, Taipei 106344, Taiwan

²Department of Computer Science and Information Engineering, National Taipei University, New Taipei City 237303, Taiwan

³Department of Information and Communication Engineering, Chaoyang University of Technology, Taichung 413310, Taiwan

Corresponding author: Yo-Ping Huang (yphuang@ntut.edu.tw)

This work was supported in part by the National Science and Technology Council, Taiwan, under Grant NSTC114-2221-E-027-128 and Grant NSTC114-2637-8-027-012.

ABSTRACT The development of the Artificial Intelligence of Things (AIoT) has created new opportunities for improving aquaculture management, particularly in shrimp hatchery operations. By integrating computer vision techniques with connected monitoring systems, AIoT technologies enable automated observation of post-larval shrimp populations. In this study, an AIoT-based computer vision system is proposed for detecting and counting post-larval shrimp in aquaculture environments. The proposed framework adopts a YOLO-based detection architecture enhanced with DarknetConv2D to improve feature extraction capability for very small shrimp objects that often appear densely distributed in hatchery conditions. Different from conventional YOLO-based detection approaches, the proposed system emphasizes an integrated AIoT deployment framework that combines lightweight detection, automated counting, edge computing, IoT communication, and remote monitoring. To evaluate the effectiveness of the proposed system, experiments were conducted under two operational scenarios, namely a waterslide scenario representing dynamic water flow conditions and a dam scenario representing stationary water environments. Quantitative results demonstrate that the proposed model achieves 99.1% detection accuracy, 94.8% of precision, 88.1% of recall, and an F1-score of 91.3%, with an mAP₅₀ of 98.6%. In addition, the model maintains a lightweight architecture with 16.2 M parameters while achieving a high inference speed of 185 FPS, enabling efficient deployment for automated monitoring systems. The proposed system demonstrates strong potential for improving counting accuracy, reducing manual labor, and supporting the development of intelligent aquaculture management systems.

INDEX TERMS Artificial Intelligence of Things (AIoT), aquaculture, post-larval shrimp detection, shrimp counting, object detection.

I. INTRODUCTION

Shrimp farming has become a crucial economic sector within aquaculture, significantly contributing to the growth of the global fishery industry [1]. Counting post-larval shrimp is an important task in shrimp hatchery operations, as it helps farmers estimate stocking density and accurately predict production capacity in grow-out ponds. However, many shrimp farmers still rely on traditional manual counting methods using visual observation with the naked eye [2], [3].

The associate editor coordinating the review of this manuscript and approving it for publication was Kit Yan Chan¹.

Because post-larval shrimp are small and often move in clusters, manual counting is highly prone to errors caused by overlapping [4], [5], adhesion, and stacking of individuals [6]. In addition, manual counting requires direct physical interaction with the larvae, which may disturb their behavior and potentially affect their health and survival [7]. These limitations not only reduce counting accuracy but also lead to inefficient use of labor and time in hatchery operations. Recent advances in artificial intelligence and computer vision have enabled automated detection and counting in aquaculture applications [8], [9]. In this study, we propose an AIoT-based computer vision system for

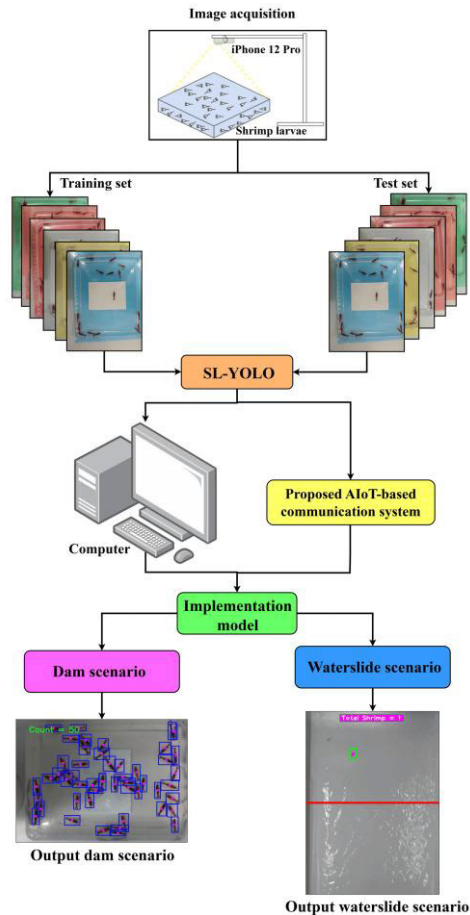


FIGURE 1. Overview architecture of the proposed system.

detecting and counting post-larval shrimp in aquaculture environments. The proposed system is designed to automate the counting process and improve operational efficiency in shrimp hatcheries. Figure 1 illustrates the overall architecture of the proposed system, including data acquisition, model training, a Raspberry Pi 5 edge computing module, a monitoring dashboard, water quality sensors and actuators, a wireless network router, and IoT communication devices. A camera captures images of post-larval shrimp in the observation area, and the trained detection model processes the images on the server, enabling the counting results and monitoring information to be visualized through a web-based dashboard. Unlike existing YOLO-based shrimp detection studies that mainly focus on improving detection accuracy under controlled imaging conditions, this study emphasizes a system-level AIoT framework for practical post-larval shrimp monitoring in aquaculture hatcheries. The novelty of the proposed work lies in the integration of an enhanced YOLO-based detector with edge-computing implementation, IoT communication, real-time counting, and web-based remote monitoring. In addition, the proposed framework is specifically adapted for detecting extremely small and densely distributed post-larval shrimp under different operational scenarios and background conditions.

The major contributions of this work are summarized as follows:

- We propose an AIoT-based computer vision system for detecting and counting post-larval shrimp in aquaculture environments. The proposed system aims to replace traditional manual counting methods that are time-consuming, labor-intensive, and prone to counting errors.
- We investigate the effect of different background colors (red, orange, green, blue, yellow, and white) on the detection performance to analyze how visual contrast influences the accuracy and precision of post-larval shrimp detection.
- We develop an enhanced YOLO-based detection framework integrated with DarknetConv2D layers, specifically designed to improve feature extraction for extremely small and densely distributed post-larval shrimp. This model-level adaptation is combined with an AIoT-oriented deployment workflow, distinguishing the proposed approach from conventional YOLO-based detection studies that primarily focus on algorithmic improvement alone.
- We demonstrate the practical deployment of the proposed system in an aquaculture environment using an off-the-shelf IoT device, enabling automated monitoring and data processing.

II. RELATED WORK

A. POST-LARVAL SHRIMP DETECTION APPROACHES

Recent advances in computer vision have enabled the development of automated detection methods for shrimp in aquaculture environments. Duan et al. [10] improved the YOLOv5 architecture by incorporating regional segmentation to handle the challenges of detecting densely populated shrimp. Their method integrated C2f modules and attention mechanisms to improve feature extraction and detection robustness. Similarly, Zhang et al. [11] proposed CAGNet, an anchor-free detection model designed for large-scale shrimp farming environments. The proposed approach demonstrated improved precision, recall, and average precision compared with conventional object detection methods. Another detection-based approach was introduced by Long et al. [12], who proposed an enhanced HRNet-based framework for post-larval shrimp detection using multi-scale feature extraction. Their method aimed to reduce information loss caused by illumination changes and occlusions between larvae. These detection-oriented approaches primarily focus on improving feature extraction and object localization accuracy, but they often face difficulties when dealing with extremely small larvae or highly dense populations.

B. POST-LARVAL SHRIMP COUNTING METHODS

Several studies have explored automated counting strategies for shrimp using different machine learning and deep learning techniques. Hu et al. [13] utilized deep learning-based density estimation to count shrimp populations. Their method

demonstrated improved computational efficiency and counting accuracy compared with traditional manual counting approaches. However, the approach requires simplified image backgrounds to accurately separate shrimp regions. Zhou et al. [14] proposed a density map regression framework called Shrimp Larvae Counting Network (SLCNet) that simultaneously estimates the number, location, and size of shrimp. The proposed method achieved high counting accuracy and demonstrated promising localization performance. Similarly, Li et al. [15] introduced a *Penaeus* Larvae Counting Strategy (PLCS) based on keypoint regression for counting *Penaeus monodon* larvae using smartphone images, enabling practical counting solutions for shrimp hatcheries without requiring specialized equipment. Other researchers have explored optical counting techniques. Qian and Duan [16] proposed a clustering-based optical counting method using masked k-means clustering and a side-window filtering approach. This method addressed challenges associated with high-density shrimp populations where traditional density estimation methods may fail due to the extremely small size of shrimp. Meanwhile, Thanasutives et al. [17] proposed M-SFANet, a hybrid neural network that combined SFANet and SegNet architectures to improve counting accuracy in crowded object scenarios.

C. AIOT-BASED AQUACULTURE MONITORING SYSTEMS

The integration of Artificial Intelligence and the Internet of Things (AIoT) has recently attracted significant attention in aquaculture monitoring systems. Hsieh et al. [18] proposed an AIoT-based shrimp counting system utilizing a scaled multilayer feature fusion network (SMILES-Net). Their system enabled remote monitoring and automated counting of shrimp, reducing manual labor and allowing faster responses to maintain shrimp health and survival. Beyond aquaculture, AIoT technologies have also been widely adopted in various counting applications. For example, Bukas et al. [4] developed an AIoT-based machine vision system for automated product counting and defect detection in manufacturing environments. These systems demonstrated the effectiveness of combining intelligent perception with connected monitoring infrastructure.

D. AIOT-BASED COUNTING AND TRACKING APPLICATIONS

AIoT technologies have also been applied to other counting and tracking tasks across different domains. Liao and Chen [19] proposed an AIoT system that integrated artificial intelligence and IoT devices to monitor pedestrian traffic at campus building entrances and exits. The system improved the efficiency of human flow monitoring and provided useful insights for facility management. Sabron et al. [20] investigated feature engineering and machine learning techniques for device-free people counting using Wi-Fi signals in IoT environments. Their study provided insights into measurement datasets, open research challenges,

and potential applications in smart buildings. In the agricultural domain, Kiobia et al. [21] explored the use of AI and IoT technologies to detect, classify, and count insect pests and beneficial insects in cotton farming systems. Furthermore, Youssef and Elshenawy [22] developed an AIoT-based traffic monitoring system using deep learning models such as convolutional neural networks and feature pyramid networks for vehicle counting and tracking from aerial video data. Their system utilized edge computing to reduce latency and enable efficient real-time processing.

Despite the significant progress achieved by previous studies in shrimp detection and counting, several limitations still remain. Many existing approaches rely on simplified experimental conditions, such as uniform background environments or controlled imaging setups, which may limit their applicability in real aquaculture scenarios. Density estimation and regression-based counting methods often struggle when the shrimp population becomes extremely dense or when individuals overlap significantly. In addition, several detection-based methods focus primarily on improving model accuracy but do not consider the integration with intelligent monitoring infrastructure required for practical deployment in aquaculture environments.

Furthermore, although AIoT-based monitoring systems have been explored in different domains, only a limited number of studies have investigated the integration of deep learning-based shrimp detection with AIoT platforms for automated monitoring and counting in aquaculture hatcheries. This limitation highlights the need for a robust computer vision system capable of accurately detecting and counting post-larval shrimp while supporting remote monitoring and intelligent aquaculture management.

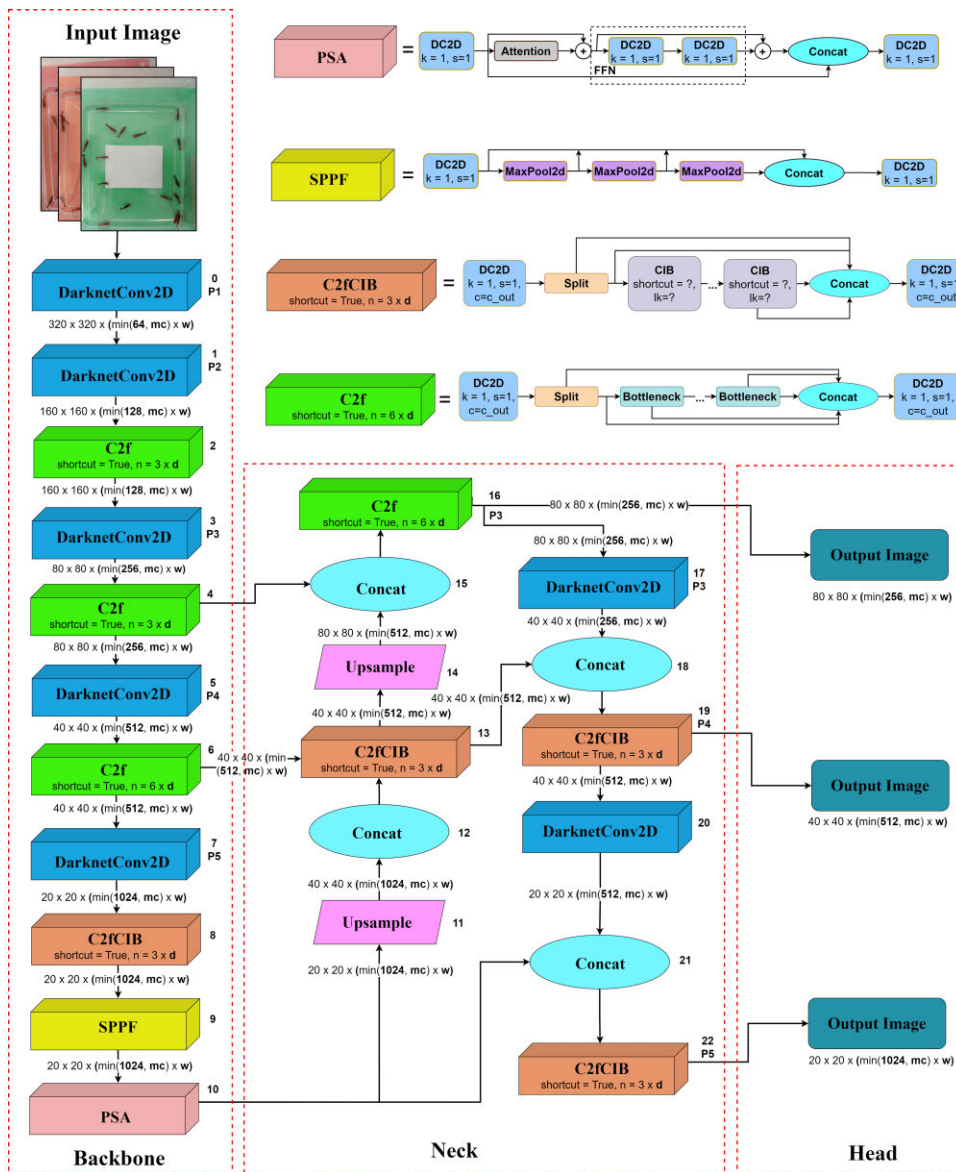
To provide a clearer overview of the existing literature, Table 1 summarizes representative studies related to shrimp detection, counting methods, and AIoT-based monitoring systems, including their datasets, methods, and reported limitations. As shown in Table 1, most existing studies focused either on detection or counting tasks independently, and only a few systems integrated computer vision with IoT infrastructure for practical aquaculture deployment. Therefore, the novelty of this study should be interpreted from both the model-level and system-level perspectives. At the model level, the YOLO-based detector is enhanced using DarknetConv2D layers to improve feature representation for very small shrimp larvae. At the system level, the proposed method is embedded into an AIoT framework that supports image acquisition, edge-based processing, IoT communication, automated counting, and remote monitoring. This integrated design addresses the limitation of prior studies that focus mainly on standalone detection models without considering deployment-oriented aquaculture monitoring requirements.

III. PROPOSED SYSTEMS

In this section, we present the proposed AIoT-based computer vision system for post-larval shrimp detection and counting.

TABLE 1. Summary of related studies on post-larval shrimp detection, Counting methods, and small object detection.

Study	Year	Task	Method	Dataset
Hu et al. [13]	2022	Counting	Density estimation CNN	Shrimp images
Duan et al. [10]	2024	Detection	YOLOv5 + segmentation	Shrimp dataset
Zhou et al. [14]	2023	Counting	Density map regression	SLCNet dataset
Li et al. [15]	2023	Counting	Keypoint regression	Smartphone images
Hsieh et al. [18]	2024	AIoT counting	SMILES-Net	Aquaculture system
Akyon et al. [23]	2022	Small object detection	SAHI (Slicing Aided Hyper Inference)	VisDrone, xView
Zhang et al. [24]	2026	Small object detection	FGUDet (distribution refinement)	UAVDT, VisDrone
Sun et al. [25]	2026	Small object detection	WE-YOLO (wavelet-based)	AI-TOD, DIOR, VisDrone
Sui et al. [26]	2026	Small object detection	Swift-YOLO (lightweight)	VisDrone, SIMD, DOTA
Zhang et al. [27]	2026	Small object detection	FPF (Focused Perception Framework)	CPE-DET

**FIGURE 2.** Overview architecture of the proposed system.

The proposed detection model is developed based on the YOLOv10 architecture and enhanced by integrating

DarknetConv2D layers to improve feature extraction for small objects such as post-larval shrimp.

A. NETWORK ARCHITECTURE

The proposed detection framework builds upon the YOLOv10 [28], [29] architecture while replacing the conventional Conv2D layers with DarknetConv2D modules to enhance feature extraction capability. Post-larval shrimp are small and often appear densely clustered in hatchery environments, which makes feature representation challenging for standard convolution layers. The DarknetConv2D module improves spatial feature extraction by strengthening channel representation while reducing information loss during down-sampling operations.

As illustrated in Figure 2, the modified architecture consists of three main components: backbone, neck, and detection head. The backbone begins with several DarknetConv2D layers that progressively extract hierarchical features from the input image while reducing spatial resolution. To enhance contextual feature learning, the architecture incorporates modules such as PSA (Position-Sensitive Attention) and SPPF (Spatial Pyramid Pooling-Fast), which expand the receptive field and improve the model's ability to capture multi-scale spatial information. In addition, feature refinement blocks such as C2f and C2fCIB are introduced to strengthen feature propagation and preserve important structural details during deep feature extraction.

The neck network performs multi-scale feature aggregation using a combination of upsampling and concatenation operations, allowing the model to fuse high-level semantic features with low-level spatial information. This feature fusion mechanism enables more robust detection of small objects, particularly when post-larval shrimp appear at different scales or partially overlap in the image. Finally, the detection head generates predictions at multiple resolution levels, producing output feature maps that correspond to different object scales. These multi-scale outputs improve the detection accuracy of very small post-larval shrimp while maintaining efficient inference performance. The modified YOLOv10 architecture shown in Figure 2 enhances feature representation and multi-scale detection capability, making it well-suited for identifying and counting post-larval shrimp in complex aquaculture environments.

B. MODIFICATION STRUCTURE OF THE YOLOV10 BACKBONE

The backbone architecture was modified to improve the detection performance for small objects, as illustrated in Figure 3. The proposed structure incorporates spatial reduction separation and channel enhancement strategies to maintain detailed structural information from the input images. These modifications allow the network to capture subtle texture patterns and structural cues of post-larval shrimp that may otherwise be lost during conventional convolutional operations. To analyze the computational complexity of the proposed architecture, floating-point operations per second (FLOPs) are used as an evaluation metric. The FLOPs of the

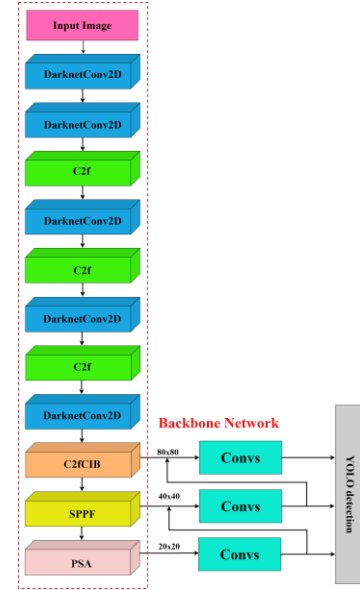


FIGURE 3. Modified backbone structure of YOLOv10 for post-larval shrimp detection.

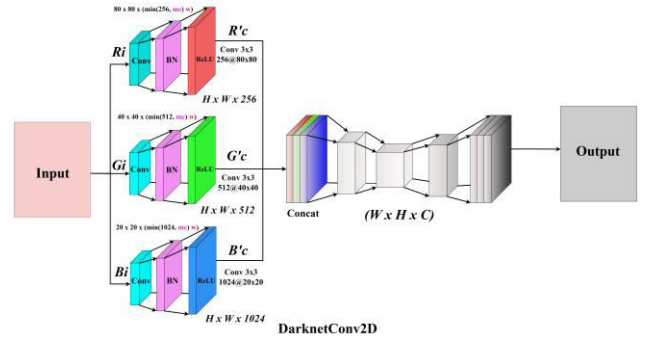


FIGURE 4. Structure of the DarknetConv2D module used in the modified YOLOv10 architecture.

convolution layers can be estimated as follows:

$$FLOPs = \sum_l^D K_l^2 \cdot M_l^2 \cdot C_l \cdot C_{l-1} \quad (1)$$

where K_l^2 is the kernel size squared, if a layer uses a $k \times k$ convolution filter. This term accounts for the number of

operations per spatial location in the kernel. D is the number of convolutional layers. M_l^2 is the size of feature map in the l_{th} convolutional layer. C_l is the number of output channels in layer l . C_{l-1} is the number of input channels in layer l .

As illustrated in Figure 4, the input image is separated into three independent branches corresponding to R_i , G_i and B_i . Each branch consists of two convolutional layers with 3×3 kernels, followed by Batch Normalization (BN) and ReLU activation. Therefore, the total computational cost can be expressed as:

$$FLOPs_{total} = FLOPs_R + FLOPs_G + FLOPs_B \quad (2)$$

where each term represents the FLOPs contribution of the corresponding RGB branch.

For the red branch, the input feature map has a dimension of $H \times W \times 64$, and the output channels are expanded to 256:

$$FLOP_{S_R} = H^2 \times 3^2 \times 64 \times 256 + H^2 \times 3^2 \times 256 \times 256 \quad (3)$$

For the green branch, the output channels are expanded to 512:

$$FLOP_{S_G} = H^2 \times 3^2 \times 64 \times 512 + H^2 \times 3^2 \times 512 \times 512 \quad (4)$$

For the blue branch, the output channels are expanded to 1024:

$$FLOP_{S_B} = H^2 \times 3^2 \times 64 \times 1024 + H^2 \times 3^2 \times 1024 \times 1024 \quad (5)$$

From a fundamental perspective, the progressive increase in channel dimensions from 256 (R), 512 (G), to 1024 (B) reflects a hierarchical feature extraction strategy. The red branch focuses on preserving fine-grained spatial details with relatively lower computational cost, while the green branch captures intermediate-level representations. The blue branch, with the highest channel capacity, is designed to learn more complex and high-level semantic features. This asymmetric channel allocation leads to a non-uniform distribution of computational cost across the three branches, where the blue branch contributes the largest portion of FLOPs. However, this design is intentional, as it enables the model to allocate more representational capacity to deeper feature abstraction, which is critical for detecting small and visually subtle objects such as post-larval shrimp.

C. PREDICTION PROCESS

The prediction process follows the standard YOLO detection mechanism with several modifications introduced to improve detection reliability for very small objects. First, the input image is resized to match the network input dimension while preserving the original aspect ratio as much as possible to minimize distortion. The image is then divided into an $S \times S$ grid, where each grid cell is responsible for detecting object whose center lies within the corresponding cell. Each grid cell predicts B bounding boxes, resulting in a total of $S \times S \times B$ predicted bounding boxes across the entire image. The confidence score of each predicted bounding box is calculated as:

$$C_i^j = P_{i,j}(\text{object}) \times IOU_{pred}^{truth} \quad (6)$$

where C_i^j is the confidence score of the j_{th} bounding box in the i_{th} grid cell. $P_{i,j}(\text{object})$ represents the probability of an object being present in the j_{th} bounding box of the i_{th} grid. If an object is detected, $P_{i,j}(\text{object}) = 1$, otherwise $P_{i,j}(\text{object}) = 0$. IOU_{pred}^{truth} is the Intersection over Union (IoU) between the predicted bounding box and the ground truth bounding box.

Bounding boxes with confidence scores lower than a predefined threshold are removed to reduce redundant detections. The remaining bounding boxes are further refined using

Non-Maximum Suppression (NMS) to eliminate overlapping predictions and retain the most representative bounding box for each detected object. The final bounding boxes and class probabilities are then used to determine the detected post-larval shrimp within the image.

D. LOSS FUNCTION

The proposed model adopts a multi-component loss function to optimize detection accuracy and training stability. The overall loss function is defined as:

$$Loss = \lambda_{bbox} L_{bbox} + \lambda_{count} L_{count} + \lambda_{cls} L_{cls} + \lambda_{conf} L_{conf} \quad (7)$$

where λ_{bbox} , λ_{count} , λ_{cls} , and λ_{conf} are weighting coefficients that balance the contributions of bounding box regression, classification, confidence, and counting losses, respectively.

The bounding box regression loss L_{bbox} follows the standard YOLO formulation, which utilizes CIOU loss to optimize spatial localization by considering overlap area, center distance, and aspect ratio consistency between predicted and ground-truth bounding boxes.

$$L_{bbox} = 1 - IoU + \frac{\rho^2(b, b^{GT})}{C^2} + \alpha v \quad (8)$$

where IoU represents the intersection over union between the predicted bounding box b and the ground-truth bounding box b^{GT} . The term $\rho^2(b, b^{GT})$ denotes the Euclidean distance between the center points of the predicted and ground-truth bounding boxes, and C is the diagonal length of the smallest enclosing box covering both boxes. The parameter v measures the consistency of aspect ratio between the predicted and ground-truth bounding boxes, while α is a positive trade-off parameter that balances the contribution of the aspect ratio term.

The counting loss is defined as:

$$L_{count} = \frac{1}{K} \sum_{k=1}^K (C_k - C_k^{GT})^2 \quad (9)$$

where C_k is the predicted number of detected shrimp in image I_k . C_k^{GT} is the corresponding ground truth count. K denotes the total number of training images.

The classification loss function L_{cls} is defined as follows:

$$L_{cls} = \frac{1}{K} \sum_{k=1}^K \|P_k - G_k\|^2 \quad (10)$$

where P_k is the predicted class probability map. G_k is the ground truth class probability map. $\|P_k - G_k\|^2$ represents the squared difference over all pixels in I_k . The confidence score loss function L_{conf} is calculated as follows:

$$L_{conf} = 1 - \frac{1}{K} \sum_{k=1}^K \frac{\sum_{x \in I_k} P_k(x) G_k(x)}{\sqrt{\sum_{x \in I_k} P_k^2(x) \sum_{x \in I_k} G_k^2(x)}} \quad (11)$$

where $\sum_{x \in I_k} P_k(x) G_k(x)$ is the sum of element-wise products of the predicted and ground truth confidence scores. $\sum_{x \in I_k} P_k^2(x) \sum_{x \in I_k} G_k^2(x)$ is the squared sum of predicted and ground truth confidence scores, which serve as normalization factors. K is the total number of training images.

IV. RESULTS AND DISCUSSION

In this section, we evaluate the performance of the proposed AIoT-based post-larval shrimp detection and counting system under different experimental conditions. The evaluation aims to analyze the detection accuracy, robustness, and computational efficiency of the proposed model when applied to aquaculture monitoring scenarios. Experimental results obtained from the proposed dataset are analyzed and compared with those reported in previous studies. The evaluation focuses on both detection accuracy and computational performance to assess the effectiveness and stability of the proposed detection framework.

A. EVALUATION METRICS

To quantitatively evaluate the performance of the proposed post-larval shrimp detection model, several commonly used object detection metrics were employed, including Precision, Recall, F1-score, and mean Average Precision (mAP). These metrics are widely used to measure detection accuracy and localization reliability in computer vision tasks. Precision represents the proportion of correctly detected post-larval shrimp among all predicted detections. Recall indicates the ability of the model to detect all relevant shrimp instances present in the dataset. The F1-score provides a balanced evaluation metric by computing the harmonic mean of precision and recall. Meanwhile, mean Average Precision (mAP) evaluates the overall detection performance by calculating the average precision across all object classes.

The evaluation metrics are defined as follows:

$$\text{Precision} = \frac{TP}{TP + FP} \times 100\% \quad (12)$$

$$\text{Recall} = \frac{TP}{TP + FN} \times 100\% \quad (13)$$

$$AP = \int_0^1 P(R) dr \quad (14)$$

$$mAP = \frac{1}{C} \sum_{i=1}^C AP_i \quad (15)$$

$$F1 - \text{score} = 2 \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \quad (16)$$

where $P(R)$ is the Precision-Recall curve and C is the number of classes. True positive samples are denoted by TP , false positive samples by FP , false negative samples by FN , and true negative samples by TN .

To provide a more appropriate evaluation for the shrimp counting task, we introduce additional metrics that directly measure the discrepancy between the predicted count and the ground truth. Unlike conventional classification accuracy, which is not suitable for object detection due to the ambiguity of true negatives, the following metrics are specifically designed for counting-based evaluation. The counting accuracy is defined as:

$$\text{Accuracy} = \left(1 - \frac{|C_{Pred} - C_{gt}|}{C_{gt}} \right) \times 100\% \quad (17)$$



FIGURE 5. Representative examples of dataset diversity and annotation quality.

where C_{Pred} and C_{gt} denote the predicted and ground truth shrimp counts, respectively. The absolute difference $|C_{Pred} - C_{gt}|$ represents the counting error, while normalization by C_{gt} provides the relative counting error. Therefore, the proposed counting accuracy is calculated as one minus the relative counting error. A higher value indicates that the predicted count is closer to the ground-truth count.

To further quantify the prediction error, we adopt Mean Absolute Error (MAE), defined as:

$$MAE = \frac{1}{N} \sum |C_{Pred} - C_{gt}| \quad (18)$$

where N is the total number of test samples. MAE measures the average absolute difference between predicted and actual counts, providing a straightforward interpretation of the counting error magnitude. In addition, we employ Root Mean Squared Error (RMSE), defined as:

$$RMSE = \sqrt{\frac{1}{N} \sum (C_{Pred} - C_{gt})^2} \quad (19)$$

RMSE penalizes larger errors more heavily due to the squared term, making it particularly useful for evaluating robustness under high-density conditions where large deviations may occur.

B. DATASET AND TRAINING

We collected six different types of datasets, totaling 3,698 images, based on background color. Each dataset group contained a similar number of post-larval shrimp but was captured using a different background color. Group 1 used a red background with 640 images, Group 2 used an orange background with 603 images, Group 3 used a yellow background with 726 images, Group 4 used a blue background with 560 images, Group 5 used a green background with 622 images, and Group 6 used a white background with 547 images (See Figure 5).

The dataset was designed to include visual diversity across different background colors to simulate different imaging conditions and contrast levels between the post-larval shrimp and the surrounding environment. In addition to background variation, the dataset also contains variations in shrimp position, orientation, spatial distribution, and density. Several samples include small objects, partially overlapping shrimp, and different object locations within the container area, which increase the detection difficulty and support the evaluation of model robustness under realistic visual conditions.

All shrimp instances were annotated using bounding boxes. The annotation process was performed manually to ensure that each visible post-larval shrimp object was labeled as accurately as possible. To maintain annotation consistency, the bounding boxes were drawn to tightly cover the visible body region of each shrimp while minimizing unnecessary background area. After the initial annotation process, the labels were reviewed to identify missing objects, inaccurate bounding boxes, duplicated labels, and inconsistent annotations. Any detected annotation errors were corrected before the dataset was used for training, validation, and testing. This verification process was conducted to improve annotation quality and reduce potential bias caused by labeling errors.

To ensure a fair and reliable evaluation, the dataset was partitioned into training, validation, and test sets using an 8:1:1 ratio. Furthermore, to prevent data leakage and ensure unbiased performance evaluation, images from the same capture session were strictly assigned to only one subset. This means that no images from the same recording sequence or acquisition condition appeared simultaneously in both the training and test sets. This session-level separation provides a more reliable evaluation of the model's ability to generalize to unseen samples. The dataset used in this study is publicly available and can be accessed at: <https://drive.google.com/file/d/1vFLLt87u1Rv56sIsGu65krXcqprbH5qg/view?usp=sharing>. We trained the model using an Nvidia GeForce RTX 4060 GPU. The model was optimized using the PyTorch framework, with a batch size of 16 to balance memory usage and computational efficiency. The initial learning rate was set to 0.05 and progressively reduced by a factor of 0.1 every 200 epochs to enhance convergence and prevent overfitting. To evaluate the impact of different environmental conditions on model performance, we conducted comparative experiments using various background settings to determine which configuration yielded the highest detection accuracy. Additionally, all input images were resized to a fixed resolution of 1200×1600 pixels for both the training and testing phases to maintain consistency and improve model generalization.

C. IMPLEMENTATION MODEL

Two operational scenarios were evaluated to assess the performance of the proposed post-larval shrimp detection and counting system. One scenario is a waterslide scenario, which evaluates the model when post-larval shrimp move rapidly

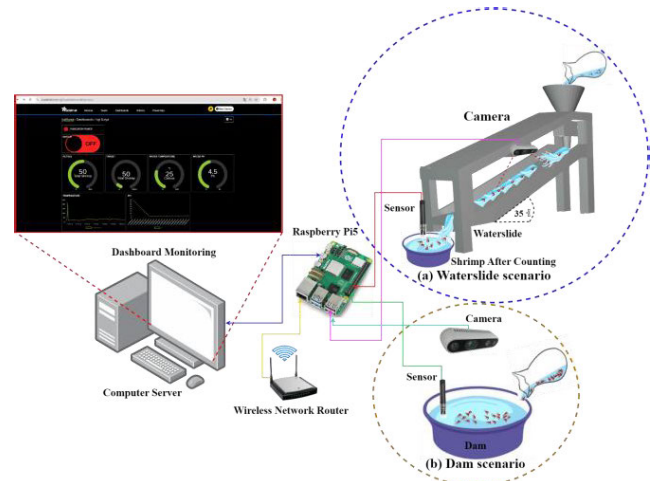


FIGURE 6. Illustration of the implementation system.

through a waterslide with flowing water. The other scenario is a dam scenario, where post-larval shrimp are placed in a container filled with stationary water. These experimental setups are given in Figure 6. A Raspberry Pi 5 was used as the embedded computing platform to support the AIoT-based monitoring system [30]. This platform enables communication between the camera module, environmental sensors, and the server database. Water quality sensors continuously monitor important environmental parameters, and the collected data are transmitted to the database through an internet connection. The dam scenario was designed to closely replicate typical shrimp hatchery conditions, where farmers traditionally count post-larval shrimp manually. In contrast, the waterslide scenario was designed to accelerate the counting process by allowing shrimp to pass through a controlled observation area where images are captured automatically.

Experimental results from both scenarios indicate that the proposed detection model is capable of accurately detecting and counting post-larval shrimp under different water conditions. Comparative analysis across multiple experimental trials with varying numbers of post-larval shrimp shows that the waterslide scenario provides reliable counting performance while enabling faster data acquisition compared to conventional manual counting methods. These results demonstrate the practical feasibility of the proposed AIoT-based monitoring system for shrimp hatchery management.

D. WATERSLIDE SCENARIO

In the waterslide scenario, several experiments were conducted using different inclination angles of the waterslide structure, specifically 20° , 25° , 30° , 35° , and 40° . Based on the experimental observations, an inclination angle of 35° was identified as the most suitable configuration, as it provided a balanced water flow that allowed post-larval shrimp to move smoothly through the observation area without excessive speed. This configuration enabled the detection system

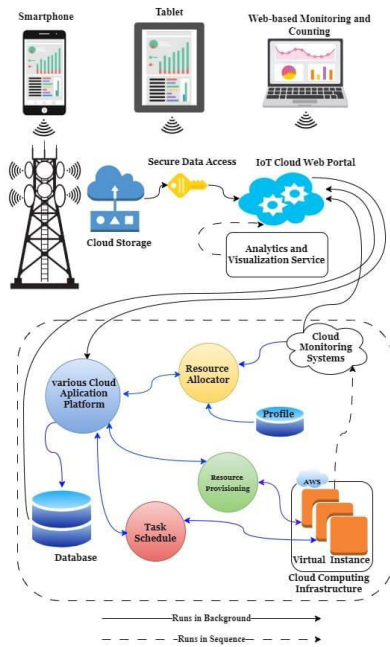


FIGURE 7. Proposed AIoT-based communication system.

to capture clear images and accurately detect and count individual post-larval shrimp.

To further improve image acquisition quality, an auxiliary lighting source was positioned near the camera to enhance visibility and reduce shadows during image capture. In addition, a white background was selected for the waterslide structure. Preliminary experiments conducted using six different background colors indicated that the white background provided the highest visual contrast with the post-larval shrimp, resulting in more reliable detection and counting performance.

To maintain stable environmental conditions during the experiments, a water quality sensor was integrated into the system to monitor critical parameters such as water temperature and pH levels. Maintaining stable water conditions is important because significant fluctuations may negatively affect the survival and behavior of post-larval shrimp. The sensor continuously records environmental data, enabling real-time monitoring of water conditions during the counting process.

Figure 7 presents the proposed AIoT-based communication architecture, which integrates edge devices, cloud infrastructure, and user interfaces to enable real-time monitoring and data analysis. At the edge level, sensor and vision data are collected and transmitted to the cloud server via HTTP/REST protocol over TCP/IP. The data transmission is performed at a frequency of approximately 1-2 Hz to ensure a balance between real-time responsiveness and network efficiency. At the cloud level, incoming data are first stored in the cloud storage system and then processed by the analytics and visualization service. This module performs detection, counting, and data aggregation before forwarding the results

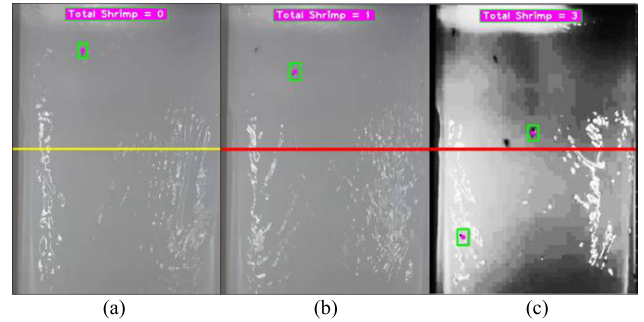


FIGURE 8. Post-larval shrimp detection results in the waterslide scenario: (a) before the counting process, (b) after the counting process, and (c) detection results with Gaussian filtering applied to enhance image contrast.

to the IoT cloud web portal. The system is supported by a cloud computing infrastructure, where virtual instances handle processing tasks, while resource provisioning and task scheduling modules dynamically manage computational workloads. In addition, a cloud monitoring system continuously tracks system performance and resource utilization to ensure stable operation. At the user level, the processed information is delivered through a web-based dashboard that can be accessed via smartphones, tablets, laptops, and desktop computers. The dashboard is implemented using lightweight web technologies, enabling real-time visualization without requiring specialized software. To evaluate system responsiveness, the end-to-end latency was measured from data acquisition at the edge device to visualization on the user interface. The system achieves an average latency of approximately 200-400 ms under normal network conditions, which is sufficient for real-time monitoring applications [31]. One of the main advantages of the proposed AIoT communication system is its lightweight data transmission and minimal storage requirements, which improve system efficiency and operational flexibility [32]. The integration of the Raspberry Pi 5 edge computing module with the monitoring dashboard enables shrimp farmers to remotely observe water quality conditions and system status.

Figure 8 presents the experimental results obtained from the waterslide scenario. In Figure 8(a), the yellow line indicates the stage before the counting process is initiated. Figure 8(b) shows the red line, which represents the stage where the system has completed the detection and counting of post-larval shrimp. To further improve detection accuracy, a Gaussian filtering technique was applied during the experiment shown in Figure 8(c). This filtering process enhances the contrast between foreground and background regions, allowing the system to more effectively distinguish individual post-larval shrimp and improve counting reliability.

E. DAM SCENARIO

In the dam scenario, post-larval shrimp were placed in a water-filled dam while a camera was positioned vertically above the dam to capture images for detection and counting,



FIGURE 9. Detection results of post-larval shrimp in the dam scenario using the proposed computer vision system.

TABLE 2. Multi-trial counting performance evaluation under varying shrimp densities.

Density (Shrimp)	Actual Count	Mean Prediction $\pm$ Std	MAE	RMSE
10	10	9.9 ± 0.3	0.2	0.4
50	50	49.5 ± 0.7	0.7	1.0
100	100	96.7 ± 1.1	3.3	3.7
200	200	193.7 ± 1.8	6.3	6.8
500	500	456.7 ± 5.6	43.3	44.5
1000	1000	879.0 ± 8.2	121	123

as illustrated in Figure 9. Several previous studies that utilized dam-based experimental setups for post-larval shrimp counting relied on density estimation methods rather than directly counting shrimp individuals from captured images [1], [18], [33], [34]. In contrast, the proposed system performs direct object detection and counting from the camera images.

To evaluate the counting capability of the proposed system, a series of experiments were conducted under varying shrimp density conditions. The evaluation includes multiple density levels, namely 10, 50, 100, 200, 500, and 1000 post-larval shrimp, as summarized in Table 2. For each density level, the experiment was repeated nine times using independent image acquisitions to ensure statistical reliability and robustness of the results. Specifically, each trial was conducted as a separate image-capturing process under the corresponding shrimp density condition. The nine repeated trials were not obtained from adjacent video frames or from a video/image sequence split frame by frame. The counting performance is assessed using the mean prediction and standard deviation, along with Mean Absolute Error (MAE) and Root Mean Squared Error (RMSE). The results indicate that the proposed system achieves highly accurate and consistent counting performance at low densities

(10-50 shrimp), as reflected by minimal error and low variance. However, as the number of shrimp increases, the counting error gradually becomes more pronounced. This performance degradation is primarily caused by increased object overlap, occlusion, and visual complexity in high-density conditions. Despite these challenges, the system maintains stable performance trends and provides reasonably accurate estimates even at higher densities.

Lighting conditions are known to significantly influence the performance of object detection systems in computer vision applications, particularly when detecting very small objects such as post-larval shrimp. Variations in illumination can affect image contrast, object visibility, and feature representation, which may ultimately impact the confidence scores and detection reliability of the model. Therefore, additional experiments were conducted to analyze the effect of image brightness variations on the detection performance of the proposed system.

In this experiment, three background colors with relatively dark visual characteristics red, orange, and yellow were selected to evaluate the robustness of the detection model under different illumination conditions. These background colors were chosen because they are commonly used in shrimp hatchery observation containers and provide varying levels of contrast with the shrimp body. For each background configuration, the brightness level of the captured images was systematically adjusted by increasing the brightness by +30% and decreasing it by -30%, while the original image brightness (0%) was used as the baseline condition. This adjustment simulates practical variations in lighting that may occur in real aquaculture environments due to different light sources, camera exposure settings, or environmental illumination changes.

The experimental results are illustrated in Figure 10, which presents the detection outputs for the three background colors under the three brightness conditions. The figure shows that higher brightness levels generally produce clearer object boundaries and stronger visual contrast between the post-larval shrimp and the background, which helps the detection model identify shrimp objects more reliably. As a result, images with increased brightness (+30%) tend to produce higher detection confidence scores and more stable bounding box predictions. In contrast, when the brightness is reduced (-30%), the overall image becomes darker and the contrast between shrimp objects and the background decreases. This reduction in contrast can make the shrimp more difficult to distinguish from the background, leading to slightly lower confidence values and occasionally less stable detections.

To further evaluate the detection performance, the proposed detection framework was compared with several YOLO-based models, including YOLOv8x, YOLOv9e, and YOLOv10x. The comparison results demonstrate that the proposed approach produces higher detection confidence values and more consistent bounding box predictions compared with the other models, with an improvement of

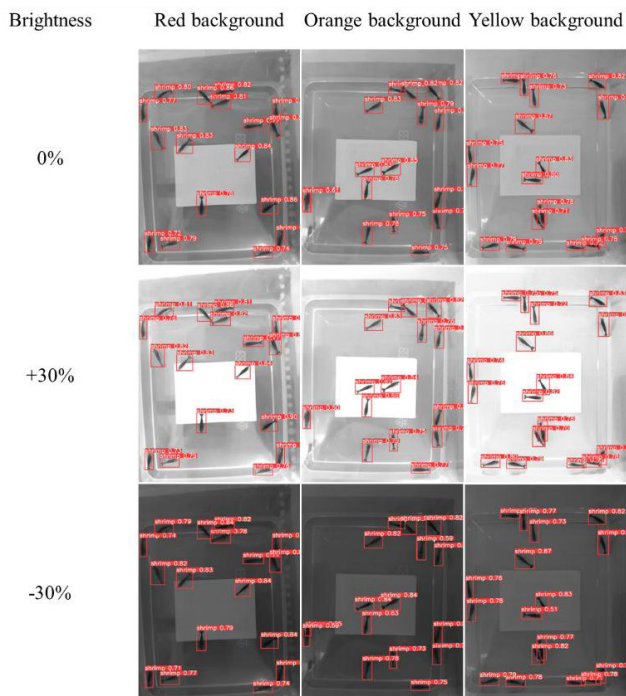


FIGURE 10. Impacts of brightness on the detection performance.

approximately 4-5%. Figure 11 presents a qualitative comparison of detection results across six different experimental groups, each corresponding to a different background color configuration used in the shrimp observation containers. The first row shows the original input images, while the second row displays the ground-truth annotations indicating the actual locations of the post-larval shrimp. The subsequent rows illustrate the detection outputs generated by YOLOv8x, YOLOv9e, YOLOv10x, and the proposed detection framework, respectively.

From the visual comparison, several important observations can be made. First, the detection results produced by the baseline models occasionally show missing detections or unstable bounding boxes, particularly in regions where the shrimp are located near the container boundaries or partially overlap with each other. This phenomenon is more noticeable in densely populated areas where multiple shrimp appear close together, which increases the difficulty of small-object detection. In contrast, the proposed framework demonstrates more stable and complete detections, successfully identifying most shrimp instances across all groups. Second, the proposed method generates more precise bounding boxes with higher confidence scores, indicating that the feature representation learned by the modified architecture is more effective for capturing the small structural characteristics of post-larval shrimp. The improvement can be attributed to the enhanced feature extraction capability introduced in the modified backbone, which helps preserve fine spatial details and improve discrimination between shrimp objects and background regions.

Another important observation from Figure 11 is the influence of background color on detection performance. Experiments reveal that images captured with lighter background colors, such as yellow, blue, and green containers, provide higher visual contrast between the shrimp body and the background. This increased contrast allows the detection model to more easily distinguish shrimp features from the surrounding environment, resulting in higher detection confidence and more accurate localization. Conversely, darker backgrounds may reduce contrast and make shrimp features less distinguishable, which can slightly affect detection reliability.

The training performance curves for precision, recall, mAP50, and mAP50-95 are presented in Figure 12. As shown in Figure 12(a), the recall performance is slightly lower than YOLOv8x but remains higher than YOLOv9e and YOLOv10x. Figure 12(b) shows that the proposed detection model achieves slightly higher precision than YOLOv10x. Furthermore, the proposed model demonstrates faster convergence during the early training stages, particularly around epoch 10. Meanwhile, Figure 12(c) and Figure 12(d) present the mAP evaluation results, indicating that the proposed model achieves competitive detection performance compared with the other YOLO variants. The experimental results from the dam scenario demonstrate that the proposed AIoT-based detection system can reliably detect and count post-larval shrimp under controlled aquaculture conditions, while maintaining stable performance across varying lighting and background environments.

F. ANALYSIS AND COMPARISON WITH STATE-OF-THE-ART MODELS

To ensure a fair comparison, all state-of-the-art models listed in Table 3 were implemented and evaluated using the same post-larval shrimp dataset and the same experimental protocol as the proposed method. The reported accuracy values were obtained from our experimental evaluation rather than being directly taken from the original publications. Specifically, the same dataset, data split, preprocessing procedure, and evaluation metric were applied to all compared models. To further evaluate the effectiveness of the proposed detection framework, its performance was compared with several state-of-the-art shrimp counting, including both counting-based approaches and detection-based approaches, and detection models implemented under the same experimental setting. The comparison results are summarized in Table 3, which reports the performance of each model in terms of accuracy, number of parameters, and computational complexity measured in FLOPs. These metrics provide a comprehensive evaluation of both detection performance and model efficiency.

For the counting-based models, methods such as CSR-Net, CAN, VGG16 (Fan's), MCNN, P2PNet, CCTrans, and Density Map primarily rely on density estimation or regression-based counting strategies. These models typically focus on estimating the number of shrimp instances

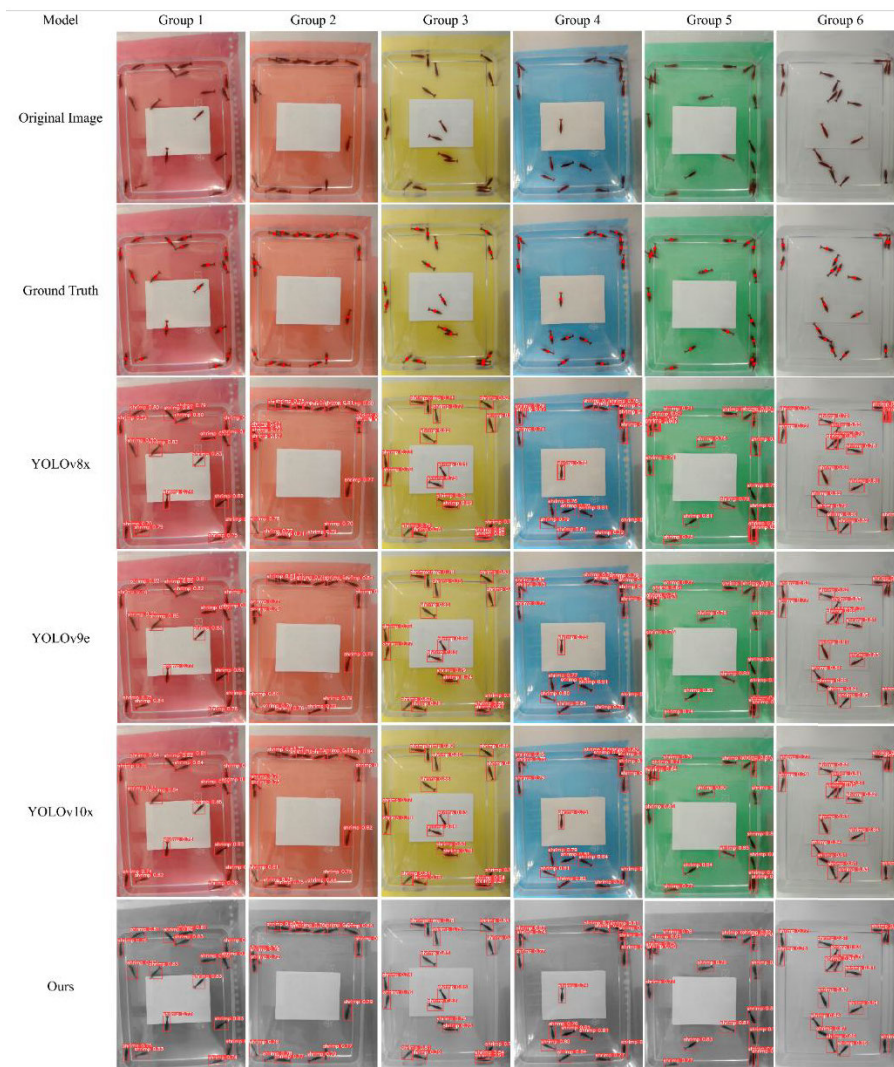


FIGURE 11. Comparison of output image detection results between the proposed detection framework and other YOLO-based models.

by predicting density maps rather than performing direct object detection. As shown in Table 3, CSRNet achieves an accuracy of 95.2%, while CAN and Density Map obtain 91.6% and 92.3%, respectively. However, these models often require relatively high computational resources, with CSRNet and Density Map reaching 165.2 GFLOPs and 173.7 GFLOPs, respectively. In addition, some models such as CCTrans exhibit extremely large parameter sizes (408.8 M), which may limit their deployment in real-time or resource-constrained aquaculture environments.

For the detection-based models, several architectures including ShrimpSeedNet, SMILES-Net, SANet, ShrimpNet, ShrimpCountNet, SmartShrimpFarm, and Mask R-CNN were evaluated. These approaches directly detect shrimp instances using object detection frameworks, which generally provide more accurate localization of individual shrimp objects. Among these models, ShrimpCountNet achieves an accuracy of 98.7%, while SMILES-Net and SANet obtain 98.0% and 97.8%, respectively. Although some models

demonstrate competitive detection accuracy, many of them require relatively large parameter sizes or higher computational complexity. Compared with our previous work, ShrimpNet [37], which primarily focused on a conventional detection framework without incorporating advanced backbone modifications or system-level integration, the proposed method introduces several significant improvements. Specifically, the proposed approach integrates a modified backbone architecture based on DarknetConv2D to enhance feature extraction capability, incorporates Gaussian filtering to improve input feature quality under varying environmental conditions, and adopts an optimized channel configuration to achieve a better balance between detection performance and computational efficiency. Furthermore, the proposed system extends beyond a standalone detection model by integrating a complete AIoT-based framework for real-time deployment and monitoring. These enhancements enable the proposed model to achieve higher detection accuracy while significantly reducing model complexity and improving

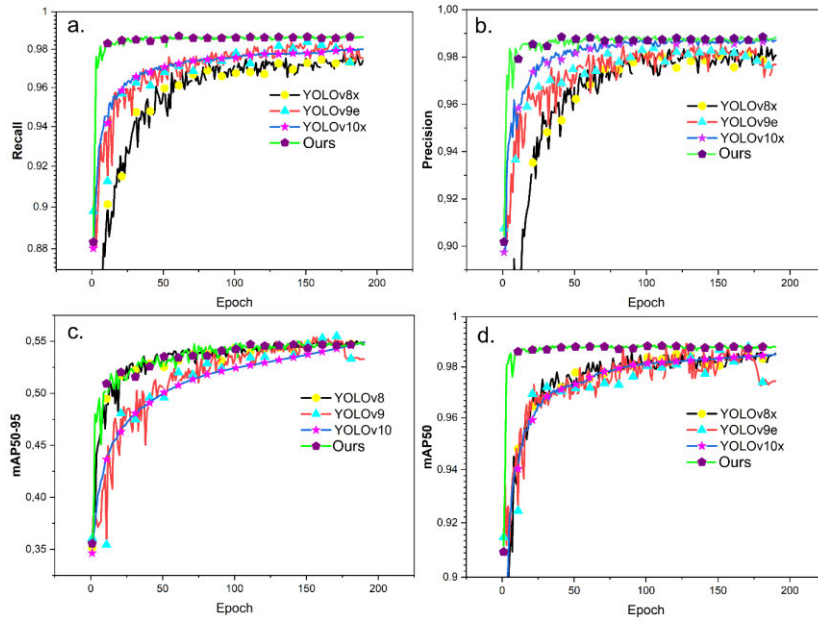


FIGURE 12. Training process curves. (a) Recall, (b) Precision, (c), mAP_{50-95} , and (d) mAP_{50} .

TABLE 3. Comparison with state-of-the-art post-larval shrimp counting and detection models.

Model	Accuracy↑ (%)	Param↓ (M)	FLOPs↓ (G)
Counting-based models			
CSRNet [35]	95.2	127.1	165.2
CAN [36]	91.6	70.8	140.7
VGG16 (Fan's) [33]	76.2	25.2	135.1
MCNN [37]	78.1	17.1	80.2
P2PNet [38]	89.0	84.3	104.8
CCTrans [33]	87.3	408.8	140.3
Density Map [10]	92.3	93.6	173.7
P2CountNet [39]	94.3	85.4	139.3
Detection-based models			
ShrimpSeedNet [33]	95.5	22.2	116.7
SMILES-Net [18]	98.0	42.4	17.7
SANet [13]	97.8	35.6	99.9
ShrimpNet [40]	94.7	2.7	12.9
ShrimpCountNet [13]	98.7	45.3	125.8
SmartShrimpFarm [10]	91.4	73.1	97.5
Mask R-CNN [41]	86.6	46.8	170.5
EMSPAN [42]	93.5	1.9	6.74
Ours	99.1	16.2	130.7

Note: All compared models were implemented and evaluated using the same dataset, data split, preprocessing procedure, and evaluation metric as the proposed method. The reported accuracy values were obtained from our experiments and were not directly taken from the original publications.

real-time performance, thereby representing a substantial advancement over the previous work.

In comparison, the proposed approach (Ours) achieves the highest detection accuracy of 99.1%, outperforming all compared models listed in Table 3. Moreover, the proposed model maintains a relatively compact parameter size of 16.2 M, which is significantly smaller than many existing models such as CSRNet (127.1 M) and CCTrans (408.8 M). Although the computational complexity of the proposed method is

moderate (130.7 GFLOPs), it provides an effective balance between detection accuracy and model efficiency. These results indicate that the proposed detection framework is capable of delivering highly accurate shrimp detection and counting performance while maintaining practical computational requirements for deployment in real-world aquaculture monitoring systems.

To evaluate the performance of the proposed detection framework, a comparative experiment was conducted using several YOLO-based architectures, including YOLOv8x [40], YOLOv9e [32], and YOLOv10x [33]. The quantitative results are presented in Table 4, which reports performance in terms of precision, recall, F1-score, mAP_{50} , number of parameters, and inference speed (FPS). These metrics provide a comprehensive evaluation of both detection accuracy and computational efficiency. As shown in Table 4, the proposed model achieves the highest detection accuracy of 99.1%, outperforming all baseline YOLO models. In addition, it achieves 94.8% precision, 88.1% recall, and 91.3% F1-score, indicating balanced detection performance. The model also obtains an mAP_{50} of 98.6%, demonstrating its effectiveness in detecting small and densely distributed post-larval shrimp. From a computational perspective, the proposed model shows significant efficiency advantages. Compared with YOLOv8x (90.0 M parameters) and YOLOv9e (53.3 M parameters), the proposed framework requires only 16.2 M parameters, making it substantially lighter. This reduction in model complexity facilitates deployment on resource-constrained edge devices commonly used in aquaculture monitoring systems.

In terms of inference speed, the proposed model achieves 185 FPS when evaluated on an NVIDIA

TABLE 4. Performance comparison of the proposed detection model with different YOLO models.

Model	Accuracy↑ (%)	Precision↑ (%)	Recall↑ (%)	F1-Score↑ (%)	mAP ₅₀ ↑ (%)	Param↓ (M)	FPS↑
YOLOv8x [43]	97.6	90.2	83.6	86.8	97.2	90.0	101
YOLOv9e [44]	97.9	90.9	86.4	89.6	95.8	53.3	48
YOLOv10x [45]	98.0	91.0	86.8	90.2	98.3	29.5	180
Ours	99.1	94.8	88.1	91.3	98.6	16.2	185

TABLE 5. Statistical validation of the proposed model across multiple runs.

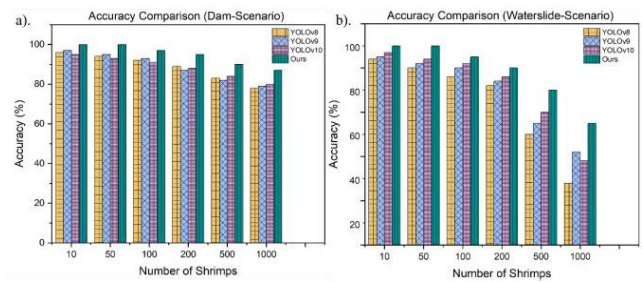
Model	Accuracy↑ (%)	Precision↑ (%)	Recall↑ (%)	F1-Score↑ (%)	mAP ₅₀ ↑ (%)
YOLOv10x [45]	98.0 ± 0.3	91.0 ± 0.5	86.8 ± 0.6	90.2 ± 0.4	98.3 ± 0.2
Ours	99.1 ± 0.2	94.8 ± 0.4	88.1 ± 0.5	91.3 ± 0.3	98.6 ± 0.2

Note: The values are reported as mean ± standard deviation over five repeated runs with different random shrimp larvae under the same dataset split, preprocessing procedure, training configuration, and evaluation protocol.

RTX 4060 GPU, slightly exceeding YOLOv10x at 180 FPS and significantly outperforming YOLOv9e at 48 FPS. However, this GPU-based performance does not directly reflect real deployment conditions. Therefore, additional benchmarking was conducted on the target edge platform, Raspberry Pi 5. The Raspberry Pi 5 benchmarking was performed using input images resized to 1200×1600 pixels, consistent with the testing configuration used in the main experiments. The model was executed using the PyTorch inference framework on Raspberry Pi 5. No quantization, pruning, or model compression was applied; therefore, the reported speed reflects the performance of the original trained model under the specified edge-device setting.

The results show that the proposed model achieves an average inference speed of 9.8 FPS on the Raspberry Pi 5, which remains sufficient for real-time monitoring applications. The observed reduction in speed compared with GPU inference is expected due to the limited computational resources of the edge device. The improved efficiency is largely attributed to the modified backbone architecture based on Darknet-Conv2D, which enhances feature extraction capability while reducing computational overhead. These results demonstrate that the proposed detection framework achieves a strong balance between high detection accuracy, lightweight model complexity, and practical real-time performance, making it suitable for real-world deployment in aquaculture environments.

To further justify the reported performance and evaluate the stability of the proposed model, an additional statistical validation was conducted using repeated experiments. The results are reported as mean ± standard deviation across five independent runs. As shown in Table 5, the proposed model achieved $99.1 \pm 0.2\%$ accuracy, $94.8 \pm 0.4\%$ precision, $88.1 \pm 0.5\%$ recall, $91.3 \pm 0.3\%$ F1-score, and $98.6 \pm 0.2\%$ mAP50. The low standard deviation indicates that the proposed method maintains stable performance across repeated evaluations and that the reported 99.1% accuracy was not obtained from a single favorable run. Compared with YOLOv10x, the proposed model consistently achieves

**FIGURE 13.** Performance comparison of YOLO-based detection models across different shrimp density levels in (a) dam scenario and (b) waterslide scenario.

higher mean performance with comparable or lower variation, demonstrating improved robustness under the same experimental protocol.

Figure 13 illustrates the accuracy comparison of different YOLO-based models under varying shrimp density conditions in both the dam scenario, as shown in Figure 13(a), and the waterslide scenario, as shown in Figure 13(b). The results show that the proposed method consistently outperforms the baseline models across all density levels. At low densities, ranging from 10 to 100 shrimp, the objects are relatively well separated, and the visual boundaries of individual shrimp remain distinguishable. Therefore, all models achieve relatively high accuracy, with the proposed method reaching near-perfect performance. As the shrimp density increases to medium levels, ranging from 200 to 500 shrimp, a gradual decline in accuracy is observed across all models. This degradation is mainly caused by increased object proximity, partial occlusion, and overlapping body regions. Under these conditions, several shrimp instances appear close to each other or partially merge visually, making it more difficult for the detector to assign separate bounding boxes to each individual object. As a result, false negatives may increase when partially occluded shrimp are missed, while false positives may occur when overlapping body parts or water reflections are incorrectly detected as separate shrimp instances.

TABLE 6. Ablation study of the proposed DarknetConv2D, and gaussian filtering.

DarknetConv2D	Gaussian Filter	Accuracy↑ (%)	Precision↑ (%)	Recall↑ (%)	F1-score↑ (%)	mAP ₅₀ ↑ (%)	Param↓ (M)	FLOPs (G)
X	X	97.0	91.0	86.8	90.2	97.8	29.5	135.0
X	✓	98.3	92.1	87.0	90.9	97.9	29.5	136.5
✓	X	98.6	93.0	87.5	91.2	97.8	20.4	132.1
✓	✓	99.1	94.8	88.1	91.3	98.6	16.2	130.7

At high-density conditions, especially from 500 to 1000 shrimp, the degradation becomes more pronounced. In this range, the detection task becomes more challenging because many shrimp occupy a limited spatial area, causing severe occlusion and visual ambiguity. The problem is more significant in the waterslide scenario than in the dam scenario. In the dam scenario, the water surface is relatively stable, and the background remains more consistent, which helps preserve object contours. In contrast, the waterslide scenario introduces additional challenges, including water motion, optical distortion, motion blur, reflection, and local turbulence. These factors reduce the visibility of small shrimp objects and make object boundaries less distinct, resulting in a larger performance drop at high density. Although the proposed model also experiences performance degradation under extreme density conditions, it maintains higher accuracy than YOLOv8x, YOLOv9e, and YOLOv10x across all tested density levels. This indicates that the proposed architecture provides stronger robustness against dense object distribution, partial occlusion, and dynamic water conditions. Nevertheless, the results also reveal the practical limitations of single-frame detection-based counting methods at extremely high densities. Future work will consider incorporating temporal information from video sequences, density-aware training, multi-frame aggregation, and improved occlusion handling to further enhance counting robustness in highly crowded aquaculture environments.

G. ABLATION STUDY

In this study, Gaussian filtering was implemented as an image preprocessing step before the input images were fed into the detection network. Specifically, the filter was applied to each RGB input image after image resizing and before pixel normalization. A Gaussian kernel size of 5×5 with a standard deviation $\sigma = 1.0$ was used. The filtering operation was applied independently to the three RGB channels to reduce high-frequency noise while preserving the main contour and texture information of post-larval shrimp objects. This preprocessing step was implemented consistently for both training and testing images in all experiments involving Gaussian filtering.

Table 6 presents the ablation study conducted to evaluate the effectiveness of the proposed DarknetConv2D module and Gaussian filtering. The baseline model without DarknetConv2D and Gaussian filtering achieves an accuracy of 97.0%, with 29.5 M parameters and 135.0 GFLOPs.

TABLE 7. Comparison model parameter.

Channel Dimensions	Param↓ (M)	FLOPs↓ (G)
(128, 128, 128)	12.8	124.1
(128, 256, 256)	13.6	125.8
(256, 256, 256)	18.5	131.5
(256, 512, 512)	19.8	132.6
(256, 512, 1024)	16.2	130.7

When Gaussian filtering is applied alone, a slight improvement in performance is observed, with accuracy increasing to 98.3% and F1-score to 90.9%. This indicates that the preprocessing step improves input image quality by reducing high-frequency noise and smoothing local intensity variations, which helps the network extract more stable features in complex visual environments. However, a small increase in computational cost is observed due to the additional filtering operation. In contrast, introducing the DarknetConv2D module results in a more significant improvement in both detection performance and computational efficiency. The accuracy increases to 98.6%, while the number of parameters is reduced from 29.5 M to 20.4 M and FLOPs decrease from 135.0 G to 132.1 G. This demonstrates that the modified backbone effectively improves feature extraction while reducing unnecessary computational overhead. The complete configuration achieves the best overall performance, with 99.1% accuracy, 94.8% precision, 88.1% recall, 91.3% F1-score, and 98.6% mAP₅₀, while maintaining the lowest parameter count (16.2 M) and reduced FLOPs (130.7 G). These results confirm that the proposed modifications not only improve detection accuracy but also enhance computational efficiency, making the model suitable for real-time deployment on resource-constrained edge devices.

To analyze the impact of channel dimension configurations on model complexity and computational efficiency, Table 7 presents a comparative evaluation of different channel settings. The selection of the asymmetric channel configuration was not intended as arbitrary empirical tuning, but was motivated by the hierarchical nature of convolutional feature extraction. In shallow branches, lower-level features such as edges, contours, texture patterns, and local intensity variations can be represented with a relatively smaller number of channels. In contrast, deeper branches require higher channel capacity to encode more abstract and discriminative semantic features, which are important for distinguishing small

TABLE 8. Performance comparison under different background colors.

Background	Precision↑ (%)	Recall↑ (%)	mAP ₅₀ ↑ (%)
Red	95.2	88.7	97.8
Orange	94.8	88.3	97.6
Yellow	95.6	89.1	97.9
Blue	93.9	87.2	97.2
Green	93.5	86.9	97.0
White	96.1	89.5	98.1

and overlapping post-larval shrimp objects under visually complex conditions. Therefore, the proposed configuration progressively increases the channel dimensions from 256 to 512 and 1024 to allocate more representational capacity to deeper feature extraction stages while avoiding unnecessary channel expansion in shallower stages.

The results show how varying the number of channels across different layers influences both the number of parameters and FLOPs. The symmetric configurations, such as (128, 128, 128) and (256, 256, 256), provide relatively balanced architectures but exhibit limitations in representational capacity because the same channel capacity is assigned to feature levels with different abstraction requirements. While the smallest configuration (128, 128, 128) achieves the lowest parameter count (12.8 M) and FLOPs (124.1 G), its reduced channel capacity may limit its ability to capture complex features required for accurate detection. Increasing the channel dimensions to (128, 256, 256) and (256, 256, 256) improves feature representation but results in a noticeable increase in computational cost. Similarly, the (256, 512, 512) configuration enhances feature extraction capability but leads to a higher parameter count (19.8 M) and FLOPs (132.6 G), indicating reduced efficiency.

The proposed asymmetric configuration (256, 512, 1024) achieves a more favorable balance between representational capacity and computational efficiency. By assigning fewer channels to shallow feature branches and more channels to deeper branches, the model can strengthen high-level feature abstraction without applying excessive computation uniformly across all stages. As shown in Table 7, this configuration achieves 16.2 M parameters and 130.7 GFLOPs, which is lower than the (256, 512, 512) setting in terms of both parameters and FLOPs. This result indicates that the proposed asymmetric configuration, together with the optimized DarknetConv2D branch design, provides a more efficient allocation of channel capacity for post-larval shrimp detection.

Table 8 presents the quantitative evaluation of the proposed model under different background color conditions, including red, orange, yellow, blue, green, and white. The performance is assessed using precision, recall, and mAP₅₀ metrics to analyze the robustness of the model against background variations. The results show that the proposed model maintains consistently high performance across all background conditions, with precision ranging from 93.5% to 96.1%,

recall from 86.9% to 89.5%, and mAP₅₀ from 97.0% to 98.1%. Backgrounds with higher contrast, such as white and yellow, achieve slightly better performance due to clearer object-background separation. In contrast, more complex backgrounds, such as green and blue, introduce minor reductions in recall, likely due to increased visual ambiguity and texture similarity with the shrimp objects. Despite these variations, the performance differences remain relatively small, indicating that the proposed model is robust and capable of generalizing effectively across diverse environmental conditions.

V. CONCLUSION

This study presented an AIoT-based computer vision system for post-larval shrimp detection and counting under two experimental scenarios. The proposed detection framework, enhanced with DarknetConv2D, improved feature extraction capability and enables efficient detection of small shrimp objects in aquaculture environments. The integration of the detection model with an AIoT platform also enabled automated monitoring and remote observation for shrimp hatchery management.

Experimental evaluations conducted under both waterslide (dynamic water flow) and dam (stationary water) scenarios demonstrated that the proposed system was capable of accurately detecting and counting post-larval shrimp under different operational conditions. Quantitative results showed that the proposed model achieves 99.1% detection accuracy, 94.8% precision, 88.1% recall, and 91.3% F1-score, with an average precision of 97.4% and mAP₅₀ of 98.6%. In addition, the model maintained a lightweight architecture with 16.2 M parameters while achieving a high inference speed of 185 FPS, making it suitable for real-time deployment in aquaculture monitoring systems. These results indicate that the proposed approach provides reliable counting performance while reducing the limitations associated with traditional manual counting methods. The main novelty of this study lies in the integration of an enhanced YOLO-based detector with an AIoT-enabled deployment framework, enabling real-time post-larval shrimp detection, counting, and remote monitoring under practical aquaculture conditions.

However, several limitations of the proposed system should be acknowledged. First, the detection performance degrades under high-density conditions due to severe object overlap and occlusion, which can affect counting accuracy. Second, in dynamic environments such as the waterslide scenario, motion blur and water-induced visual distortion introduce additional challenges for stable detection. Third, environmental factors such as water turbidity may further impact feature visibility and detection robustness. In terms of real-world deployment, the system also faces challenges related to limited computational resources on edge devices, which result in reduced inference speed compared to GPU-based evaluation, as well as potential network latency affecting real-time data transmission.

For future work, further research will focus on improving detection robustness under more complex environmental conditions, such as varying lighting levels, higher shrimp densities, and different water turbidity levels. In addition, the system can be extended by integrating edge AI acceleration and advanced tracking algorithms to improve counting stability in highly dynamic environments. Expanding the dataset with more diverse aquaculture conditions and deploying the system in large-scale shrimp hatcheries will also be explored to further validate its practicality for real-world smart aquaculture applications.

REFERENCES

- [1] Y. Qu, S. Jiang, D. Li, P. Zhong, and Z. Shen, "SLCOBNet: Shrimp larvae counting network with overlapping splitting and Bayesian DM count loss," *Biosystem Eng.*, vol. 244, pp. 200–210, Jun. 2024.
- [2] L. Zhang, X. Zhou, B. Li, H. Zhang, and Q. Duan, "Automatic shrimp counting method using local images and lightweight YOLOv4," *Biosyst. Eng.*, vol. 220, pp. 39–54, Aug. 2022.
- [3] S. Kim, S. Song, Y. Lee, Y. Lee, Y.-M. Lee, K.-J. Cho, and K.-J. Lee, "Improvement of growth, lipid metabolism and innate immune response in Pacific white shrimp (*Penaeus vannamei*) post-larvae through enrichment of live feeds with schizochytrium sp., taurine and inosine monophosphate," *PLoS ONE*, vol. 21, no. 3, pp. 1–16, Mar. 2026.
- [4] C. Bukas, F. Albrecht, M. S. Ur-Rehman, D. Popek, M. Patalan, J. Pawłowski, B. Wecker, K. Landsch, T. Golan, T. Kowalczyk, M. Piraud, and S. S. W. Ende, "Robust deep learning based shrimp counting in an industrial farm setting," *J. Cleaner Prod.*, vol. 468, pp. 1–9, Aug. 2024.
- [5] G. Dai, L. Hu, J. Fan, S. Yan, and R. Li, "A deep learning-based object detection scheme by improving YOLOv5 for sprouted potatoes datasets," *IEEE Access*, vol. 10, pp. 85416–85428, 2022.
- [6] Y. Liu, Y. Li, Y. Ma, K. Qu, J. Song, B. Tan, H. Liu, Q. Yang, S. Zhang, and S. Xie, "Dietary methionine requirement of postlarval shrimp (*litopenaeus vannamei*): Based on growth performance, antioxidant capacity, and lipid metabolism," *Aquaculture*, vol. 603, Jun. 2025, Art. no. 742424.
- [7] Z. Zhou, J. Lu, P. Zhan, and J. Xiong, "Postlarval shrimp-associated microbiota and underlying ecological processes over AHPND progression," *Microorganisms*, vol. 13, no. 4, p. 720, Mar. 2025.
- [8] Z. Yang, Y. Li, Q. Han, H. Wang, C. Li, and Z. Wu, "A method for tomato ripeness recognition and detection based on an improved YOLOv8 model," *Horticulturae*, vol. 11, no. 1, pp. 1–21, Dec. 2024.
- [9] Y.-P. Huang, T.-H. Wang, and H. Basanta, "Using fuzzy mask R-CNN model to automatically identify tomato ripeness," *IEEE Access*, vol. 8, pp. 207672–207682, 2020.
- [10] H. Duan, J. Wang, Y. Zhang, X. Wu, T. Peng, X. Liu, and D. Deng, "Shrimp larvae counting based on improved YOLOv5 model with regional segmentation," *Sensors*, vol. 24, no. 19, p. 6328, Sep. 2024.
- [11] G. Zhang, Z. Shen, D. Li, P. Zhong, and Y. Chen, "CAGNet: An improved anchor-free method for shrimp larvae detection in intensive aquaculture," *Aquaculture Int.*, vol. 32, no. 5, pp. 6153–6175, Mar. 2024.
- [12] L. Wu, Y. Yang, Y. Tang, L. Hu, J. Zhang, J. Xu, Y. Yang, W. Yang, and L. Jiang, "An improved shrimp larvae counting algorithm based on HRNET," in *Proc. IET Int. Conf. Artif. Intell. Adv. Manuf.*, vol. 2023, Brussels, Belgium, Dec. 2023, pp. 98–104.
- [13] W.-C. Hu, L.-B. Chen, M.-H. Hsieh, and Y.-K. Ting, "A deep-learning-based fast counting methodology using density estimation for counting shrimp larvae," *IEEE Sensors J.*, vol. 23, no. 1, pp. 527–535, Jan. 2023.
- [14] C. Zhou, G. Yang, L. Sun, S. Wang, W. Song, and J. Guo, "Counting, locating, and sizing of shrimp larvae based on density map regression," *Aquaculture Int.*, vol. 32, no. 3, pp. 3147–3168, Nov. 2023.
- [15] X. Li, R. Liu, Z. Wang, G. Zheng, J. Lv, L. Fan, Y. Guo, and Y. Gao, "Automatic penaeus monodon larvae counting via equal keypoint regression with smartphones," *Animals*, vol. 13, no. 12, p. 2036, Jun. 2023.
- [16] K. Qian and H.-C. Duan, "Optical counting platform of shrimp larvae using masked k-means and a side window filter," *Appl. Opt.*, vol. 63, no. 6, pp. 7–15, Nov. 2023.
- [17] P. Thanasutives, K.-I. Fukui, M. Numao, and B. Kijisirikul, "Encoder-decoder based convolutional neural networks with multi-scale-aware modules for crowd counting," in *Proc. 25th Int. Conf. Pattern Recognit. (ICPR)*, Milan, Italy, Jan. 2021, pp. 2382–2389.
- [18] Y.-K. Hsieh, J.-W. Hsieh, W.-C. Hu, and Y.-C. Tseng, "AIoT-based shrimp larvae counting system using scaled multilayer feature fusion network," *IEEE Internet Things J.*, vol. 11, no. 22, pp. 36438–36451, Nov. 2024.
- [19] H.-C. Liao and W.-Y. Chen, "Smart AIoT devices for entrance people counting," in *Proc. 8th Int. Conf. Imag., Signal Process. Commun. (ICISPC)*, Jul. 2024, pp. 89–93.
- [20] I. Sobron, J. Del Ser, I. Eizmendi, and M. Velez, "Device-free people counting in IoT environments: New insights, results, and open challenges," *IEEE Internet Things J.*, vol. 5, no. 6, pp. 4396–4408, Dec. 2018.
- [21] D. O. Kiobia, C. J. Mwitwa, K. G. Fue, J. M. Schmidt, D. G. Riley, and G. C. Rains, "A review of successes and impeding challenges of IoT-based insect pest detection systems for estimating agroecosystem health and productivity of cotton," *Sensors*, vol. 23, no. 8, pp. 1–20, Apr. 2023.
- [22] Y. Youssef and M. Elshenawy, "Automatic vehicle counting and tracking in aerial video feeds using cascade region-based convolutional neural networks and feature pyramid networks," *Transp. Res. Rec., J. Transp. Res. Board*, vol. 2675, no. 8, pp. 304–317, Mar. 2021.
- [23] F. C. Akyon, S. O. Altinuc, and A. Temizel, "Slicing aided hyper inference and fine-tuning for small object detection," in *Proc. IEEE Int. Conf. Image Process. (ICIP)*, Bordeaux, France, Oct. 2022, pp. 966–970.
- [24] J. Zhang, M. Gao, D. Zhao, Y. Zhang, and H. Wang, "Fine-grained distribution refinement for small object detection in UAV images," *IEEE Internet Things J.*, vol. 13, no. 8, pp. 17790–17800, Apr. 2026.
- [25] J. Sun, Y. Guo, X. Zhi, X. Wu, J. Wang, Z. Xu, B. Pu, and Z. Gao, "WE-YOLO: Wavelet enhanced YOLO for small object detection in optical aerial remote sensing images," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 19, pp. 1–17, 2026.
- [26] Q. Sui, Y. Hosoda, and J.-H. Lee, "Swift-YOLO: A lightweight architecture for small target detection in drone optical sensors," *IEEE Sensors J.*, vol. 26, no. 3, pp. 4212–4224, Feb. 2026.
- [27] H. Zhang, H. Liu, B. Biekezati, R. Du, Y. Cao, K. Zhang, T. Niu, and Y. Wang, "FPF: A focused perception framework for small defect identification in complex power scenarios," *IEEE Trans. Ind. Informat.*, vol. 22, no. 4, pp. 2793–2804, Apr. 2026.
- [28] H. Sun, G. Yao, S. Zhu, L. Zhang, H. Xu, and J. Kong, "SOD-YOLOv10: Small object detection in remote sensing images based on YOLOv10," *IEEE Geosci. Remote Sens. Lett.*, vol. 22, pp. 1–5, 2025.
- [29] X. Ji, Z. Yue, H. Yang, and Z. Zhang, "Infrared image classification and detection algorithm for power equipment based on improved YOLOv10," *IEEE Access*, vol. 12, pp. 184976–184988, 2024.
- [30] L. L. Bello, R. Mariani, S. Mubeen, and S. Saponara, "Recent advances and trends in on-board embedded and networked automotive systems," *IEEE Trans. Ind. Informat.*, vol. 15, no. 2, pp. 1038–1051, Feb. 2019.
- [31] J. Shi, X. Chen, Y. Zhang, X. Chen, and C. Pan, "Joint optimization of task offloading and resource allocation in satellite-assisted IoT networks," *IEEE Internet Things J.*, vol. 11, no. 21, pp. 34337–34348, Nov. 2024.
- [32] K. W. Chen, A. S. Abdullah, M. A. Sarijari, R. A. Rashid, A. R. A. M. Yunus, N. M. Noor, M. A. Rosli, and N. M. Yasin, "Artificial intelligence of things real-time fish counting system," in *Proc. IEEE Int. Conf. Adv. Telecommun. Netw. Technol. (ATNT)*, Johor Bahru, Malaysia, Sep. 2024, pp. 1–4.
- [33] D. Liu, B. Xu, Y. Cheng, H. Chen, Y. Dou, H. Bi, and Y. Zhao, "Shrimpsed-Net: Counting of shrimp seed using deep learning on smartphones for aquaculture," *IEEE Access*, vol. 11, pp. 85441–85450, 2023.
- [34] G. Zhang, T. Liao, Y. Chen, P. Zhong, Z. Shen, and D. Li, "Efficient one-stage detection of shrimp larvae in complex aquaculture scenarios," *Artif. Intell. Agricult.*, vol. 15, no. 2, pp. 338–349, Jun. 2025.
- [35] Y. Li, X. Zhang, and D. Chen, "CSRNet: Dilated convolutional neural networks for understanding the highly congested scenes," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Salt Lake City, UT, USA, Jun. 2018, pp. 1091–1100.
- [36] W. Liu, M. Salzmann, and P. Fua, "Context-aware crowd counting," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Long Beach, CA, USA, Jun. 2019, pp. 5094–5103.
- [37] Y. Zhang, D. Zhou, S. Chen, S. Gao, and Y. Ma, "Single-image crowd counting via multi-column convolutional neural network," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Long Beach, CA, USA, Jun. 2016, pp. 589–597.

- [38] Q. Song, C. Wang, Z. Jiang, Y. Wang, Y. Tai, C. Wang, J. Li, F. Huang, and Y. Wu, "Rethinking counting and localization in crowds: A purely point-based framework," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Montreal, QC, Canada, Oct. 2021, pp. 3345–3354.
- [39] T.-H. Dang, F. D. S. Neto, X. Li, B. Smithers, E. Mager, and M. Acevedo, "Size-adaptive density map estimation and multiscale feature extraction network for robust prawn postlarvae counting," *IEEE Trans. AgriFood Electron.*, vol. 1, no. 1, pp. 292–305, Mar. 2026.
- [40] Y.-P. Huang, S. Kshetrimayum, and F. Eika Sandnes, "ShrimpNet: A lightweight model for efficient detection and counting of shrimp larvae in dynamic aquaculture environments," *IEEE Trans. Instrum. Meas.*, vol. 74, pp. 1–13, 2025.
- [41] B. Xu, W. Wang, G. Falzon, P. Kwan, L. Guo, G. Chen, A. Tait, and D. Schneider, "Automated cattle counting using mask R-CNN in quadcopter vision system," *Comput. Electron. Agricult.*, vol. 171, pp. 1–12, Apr. 2020.
- [42] P. N. P. Hai, B. B. Quoc, and T. Hoang, "EMSPAN: Efficient multi-scale pyramid attention network for object counting under size heterogeneity and dense scenarios," *IEEE Access*, vol. 13, pp. 17945–17962, 2025.
- [43] M. Yaseen, "What is YOLOv8: An in-depth exploration of the internal features of the next-generation object detector," 2024, *arXiv:2409.07813*.
- [44] C.-Y. Wang, I.-H. Yeh, and H.-Y. Mark Liao, "YOLOv9: Learning what you want to learn using programmable gradient information," 2024, *arXiv:2402.13616*.
- [45] A. Wang, H. Chen, L. Liu, K. Chen, Z. Lin, J. Han, and G. Ding, "YOLOv10: Real-time end-to-end object detection," 2024, *arXiv:2405.14458*.



YO-PING HUANG (Fellow, IEEE) received the Ph.D. degree in electrical engineering from Texas Tech University, Lubbock, TX, USA.

He was the President of the National Penghu University of Science and Technology, Penghu, Taiwan. He was a Professor and the Dean of Research and Development, the Dean of the College of Electrical Engineering and Computer Science, and the Department Chair with Tatung University, Taipei. He is currently a Chair Professor with the Department of Electrical Engineering, National Taipei University of Technology, Taipei, Taiwan, where he was the Secretary General. He is also the President of Chinese Automatic Control Society, Taiwan, and the Vice President of Publication in Asian Control Association. His current research interests include deep learning modeling, intelligent control, machine learning, and the AIoT systems design.



IRGI SURYA (Member, IEEE) received the B.Ed. degree in electrical engineering education from the Faculty of Engineering and Industrial Education, Universitas Pendidikan Indonesia. He is currently pursuing the Master of Science degree with the Department of Electrical Engineering, National Taipei University of Technology, Taipei, Taiwan.

His current research interests include the Internet of Things for agriculture and aquaculture, computer vision and image processing, artificial intelligence, machine learning, deep learning, automation industry, control systems, and the AIoT systems design.

...